NizSystem: Integrated Execution, Debugging, and Memory Management Framework

Lab 1-5 Integration Project Report

Student Name: Nizaul Rahman
Mayur Kamble
Roll Number: 2025MCS2099, 2025MCS2116

February 3, 2026

Abstract

This report details the design and implementation of **NizSystem**, a unified execution framework that integrates a Shell, Compiler, Virtual Machine (VM), Garbage Collector (GC), and Debugger. Unlike standalone components, NizSystem manages the complete lifecycle of a program as a first-class entity. It features a custom scripting language with strict scoping (Lab 3), a stack-based VM (Lab 4) with automatic Mark-and-Sweep garbage collection (Lab 5), and a robust shell interface (Lab 1) that coordinates process management, compilation, and interactive debugging. The system successfully passes rigorous stress testing, including deep recursion, memory pressure, and syntax robustness checks.

Contents

1 Introduction

The objective of this project was to move beyond isolated lab implementations and build a coherent system where components depend on one another. The *NizSystem* treats a user program not just as code, but as a managed entity with a Process ID (PID), memory state, and execution context.

The system is composed of distinct but tightly coupled layers:

1. **Lab 1 (Shell):** The kernel/OS layer managing PIDs and user interaction.
2. **Lab 2 (Debugger):** External ELF debugging using `ptrace`.
3. **Lab 3 (Compiler & Scopes):** Parsing, AST generation, and Symbol Table management.
4. **Lab 4 (Virtual Machine):** A stack-based execution engine with safety checks.
5. **Lab 5 (Memory Manager):** A Mark-and-Sweep Garbage Collector.

2 System Architecture

2.1 1. Lab 1: The Shell (Process Manager)

The shell (`nizshell.c`) acts as the orchestrator. It does not merely execute binaries; it manages the `Process` table.

- **Process Table:** A fixed-size array (`procs[MAX_PROCS]`) that holds the state (`PROC_FREE`, `PROC_STOPPED`) and the VM context for each program.
- **Lifecycle Management:**
  - `submit <file>`: Compiles code and allocates a PID (e.g., PID 1).
  - `run <pid>`: Resumes the VM execution for that PID.
  - `kill <pid>`: Frees the process slot and VM memory.
- **Robustness:** Handles zombie processes via `sigchld_handler` and supports standard features like pipelines (`|`) and redirection.

2.2 2. Lab 3: Compiler & Symbol Table

The compiler is the bridge between user logic and VM execution. It implements strict semantic rules and scope management.

- **Symbol Table Logic:**
  - **Local Variables:** Managed via a stack-based symbol table. Variables declared inside blocks ({ }) shadow global variables. The compiler resolves these to `OP_LOAD_L` (Load Local) offsets relative to the Frame Pointer.
  - **Global Variables:** Variables declared at the top level are resolved to `OP_LOAD_G` (Load Global) indices.
- **Semantic Analysis (Strict Mode):** The system enforces explicit variable declaration.

```
// This triggers a Compile Error
x = 10;
// This is Valid
var x = 10;

```

- **AST Generation:** On submission, the system prints the Abstract Syntax Tree (AST) to visualize the parsing hierarchy, verifying that operator precedence and nesting are handled correctly.

2.3 3. Lab 4: The Virtual Machine (Execution Engine)

The VM is a stack machine designed for safety and observability.

- **Architecture:** Registers include PC (Program Counter), SP (Stack Pointer), and FP (Frame Pointer).
- **Safety Checks:**
  - **Stack Overflow/Underflow:** Prevents corruption during deep nesting.
  - **Division by Zero:** Traps runtime math errors gracefully.
  - **Instruction Bounds:** Halts execution if PC exceeds program size.
- **Object System:** All values on the stack are pointers to `Object` structs, enabling the Garbage Collector to track references.

2.4 4. Lab 5: Memory Management (Garbage Collector)

The system implements a **Mark-and-Sweep GC** to manage the heap automatically.

- **Allocation Strategy:** `vm_allocate` checks for free slots. If the heap is full, it triggers `vm_gc()`.
- **Mark Phase:** Traverses the VM Stack (Roots) and marks all reachable objects.
- **Sweep Phase:** Iterates through the object pool, reclaiming unmarked objects and returning them to the `OBJ_FREE` state.
- **Tools:**
  - `memstat <pid>`: Reports current heap usage.
  - `leaks <pid>`: Verifies active objects (useful for leak detection).

3 Debugging Features (Lab 2 & Integration)

3.1 Internal VM Debugger

Accessed via `debug <pid>`, this tool hooks directly into the VM struct.

- **Breakpoints:** `break 0x05` sets a flag in the `vm->breakpoints` array.
- **Stepping:** `step` executes one opcode at a time, printing the stack state.
- **Inspection:** `regs` shows PC/SP/FP; `stack` shows current values.
- **Post-Mortem:** Can reset the PC to 0 to restart debugging a terminated process.

3.2 External ELF Debugger (Lab 2)

Accessed via `edebug <binary>`, this tool uses Linux `ptrace` APIs.

- **Features:** Software breakpoints (injecting `0xCC`), register inspection (`PTRACE_GETREGS`), and memory examination (`x /n 0xADDR`).
- **Robustness:** Handles trap signals and correctly steps over breakpoints.

4 Features Matrix

Lab	Feature	Status
Lab 1	Pipelines, Redirection, Backgrounding	**Supported**
(Shell)	Zombie Process Reaping, History	**Supported**
Lab 2	External Debugger (ptrace), Memory Dump	**Supported**
(Debugger)	Breakpoints, Stepping, Regs	**Supported**
Lab 3	Symbol Table (Scopes), Strict Variables	**Supported**
(Compiler)	AST Generation, Syntax Error Reporting	**Supported**
Lab 4	Stack VM, Runtime Safety (DivZero)	**Supported**
(VM)	Integrated Debugger Hooks	**Supported**
Lab 5	Mark-and-Sweep GC, Auto-Trigger	**Supported**
(Memory)	Leak Detection, Heap Stats	**Supported**

Table 1: System Feature Matrix

5 Testing and Verification

The system was validated using a custom Python test suite (`generate_and_test.py` and `stress_test.py`).

- **Syntax Robustness:** Verified recovery from 10+ types of syntax errors (e.g., missing semicolons, bad keywords).
- **Logic Validation:** Validated algorithms like Factorial, Fibonacci, and GCD.
- **Stress Tests:**
  - **Deep Nesting:** 100 levels of {} blocks (Passed).
  - **GC Pressure:** 10,000 allocations in a loop triggered auto-GC successfully (Passed).
  - **Compiler Reset:** Verified line numbers reset correctly between submissions (Passed).

6 Conclusion

The NizSystem fulfills the integration requirements by ensuring that the Debugger relies on the VM, the VM relies on the Compiler, and the Memory Manager monitors the VM. Removing any component would break the lifecycle management, demonstrating a truly unified execution framework.